

1

Web Application & Deployment Engineer

พัฒนาเว็บแอปพลิเคชันและนำขึ้น Cloud



Interactive Web Development



Cloud Deployment



2

Overview

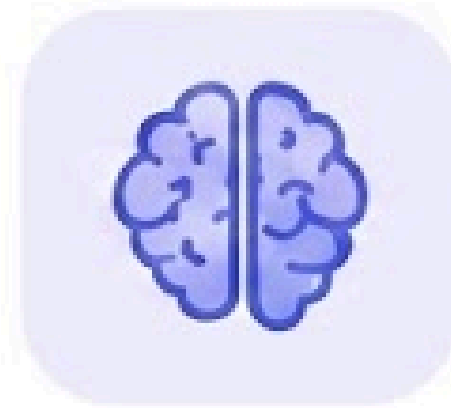
เว็บแอปพลิเคชันสำหรับจำแนกสายพันธุ์แมว 66 สายพันธุ์
ใช้งานง่าย รวดเร็ว และแม่นยำ



อัปโหลดภาพ
หรือสแกนผ่านกล้อง



ตรวจสอบคุณภาพภาพ
(ความเบลอ/ความสว่าง)



โมเดล AI
ทำนายสายพันธุ์

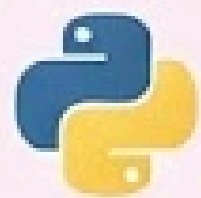


แสดงผล Top-5
พร้อมข้อมูลสายพันธุ์

เทคโนโลยีที่ใช้



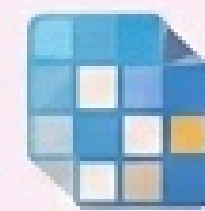
Streamlit



Python



OpenCV



skimage / NumPy



PyTorch
(โมเดล AI)

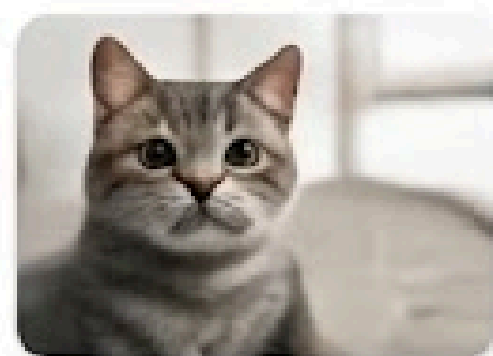
3

Interactive Web Development

1) Image Quality Audit

ตรวจสอบคุณภาพของภาพก่อนส่งเข้าโมเดล

- ✓ ตรวจสอบความคมชัด
(Laplacian Variance)



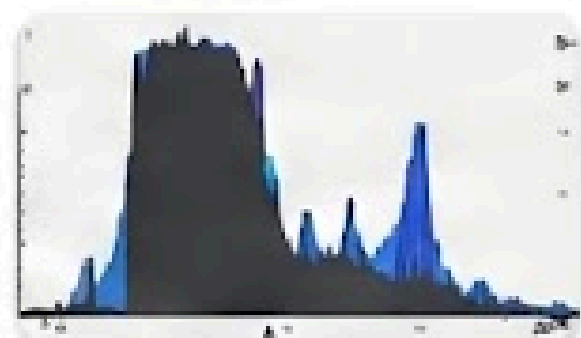
$\text{Var} \geq 100$

ภาพคมชัด

$\text{Var} \leq 100$

ภาพเบลอเกินไป

- ✓ ตรวจสอบความสว่างเฉลี่ย
(Mean Brightness)



$50 \leq \text{Brightness} \leq 200$

ความสว่างเหมาะสม

นอกช่วงนี้
สว่าง/มืดเกินไป

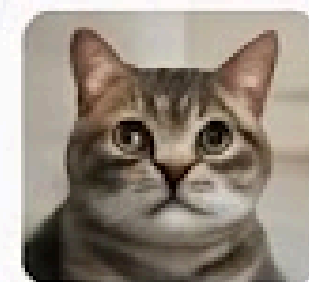
ตัวอย่างการแจ้งเตือน



ภาพเบลอเกินไป
กรุณาถ่ายใหม่



ภาพสว่างเกินไป
กรุณาปรับแสง



คุณภาพภาพดี
พร้อมวิเคราะห์

4

Interactive Web Development

2) Multi-Input Inference



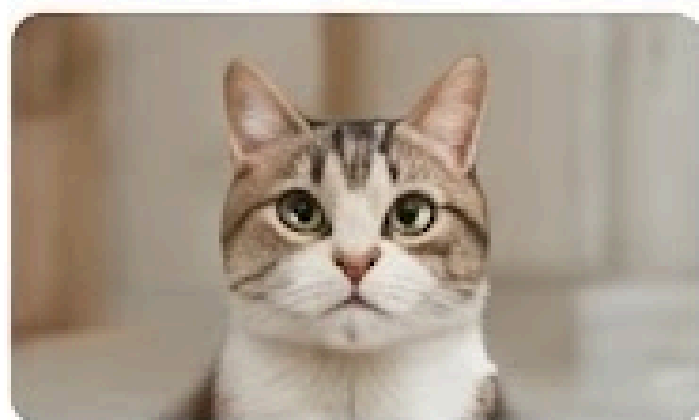
อัปโหลดไฟล์ภาพ

รองรับไฟล์ .jpg, .jpeg, .png

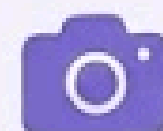


Drag and drop file here
หรือกดเลือกไฟล์

Browse files

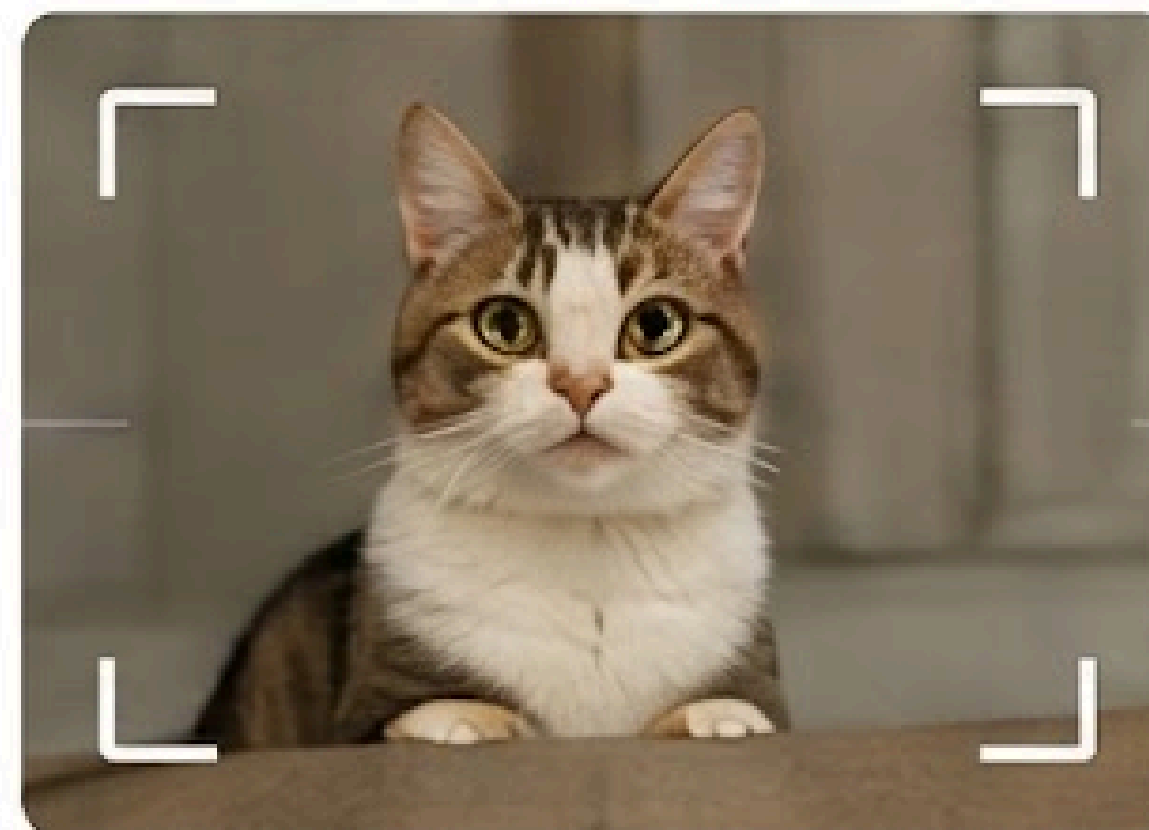


ตัวอย่างภาพที่อัปโหลด



สแกนสดผ่านกล้อง

ใช้งานผ่านกล้องสมาร์ทโฟน / เว็บแคม



เริ่มต้นสแกน

5

ตัวอย่างหน้าจอเว็บแอปพลิเคชัน

📷 ภาพที่ส่งเข้าวิเคราะห์



🔍 ผลการตรวจสอบคุณภาพ
ภาพ (Quality Audit)

Laplacian Var: 150.9 (คมชัด ผ่านเกณฑ์ ✅)

Brightness: 144.5 (แสงปกติ ผ่านเกณฑ์ ✅)

🌟 สายพันธุ์ที่ตรวจพบอันดับ 1

🐱 **Munchkin**

ความมั่นใจ 26.9%

- 📍 **ถิ่นกำเนิด:** สายพันธุ์แมวสากล
- 💡 **บุคลิกและเอกลักษณ์:** มีเอกลักษณ์เฉพาะตัว สดใสร่าเริง และมีความเฉลียวฉลาดตามธรรมชาติของสายพันธุ์

📊 5 อันดับความเป็นไปได้สูงสุด (Top 5)

#1 🐱 Munchkin 26.91%

#2 🐱 Cymric 11.38%

#3 🐱 British Shorthair 4.70%

#4 🐱 Kurilian Bobtail 3.31%

#5 🐱 European Shorthair 3.30%

6

Cloud Deployment

นำแอปพลิเคชันขึ้น Streamlit Community Cloud

ขั้นตอนการ Deploy

- 1 เตรียมโค้ดและไฟล์โมเดล (cat_breed_model.pth)
- 2 อัปโหลดไปยัง GitHub Repository
- 3 เชื่อมต่อกับ Streamlit Community Cloud
- 4 ตั้งค่า Environment และไฟล์ requirements.txt
- 5 Deploy และรับ URL สาธารณะ

โครงสร้างโปรเจกต์

```
cat-breed-classifier/  
|— app.py  
|— cat_breed_model.pth  
|— requirements.txt  
|— utils.py  
|— data/  
|— README.md
```



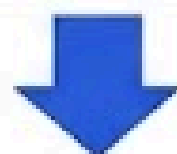
7

การเชื่อมต่อโมเดลที่บีบอัดแล้ว

ใช้ไฟล์น้ำหนักโมเดลที่ผ่านการบีบอัดแบบ FP16

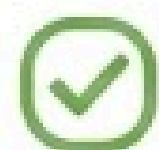
ไฟล์ต้นฉบับ (FP32)

cat_breed_model.pth
114 MB



ไฟล์บีบอัด (FP16)

cat_breed_model.pth
~57 MB



เชื่อมต่อและโหลดโมเดลในแอป

การโหลดโมเดลในแอป

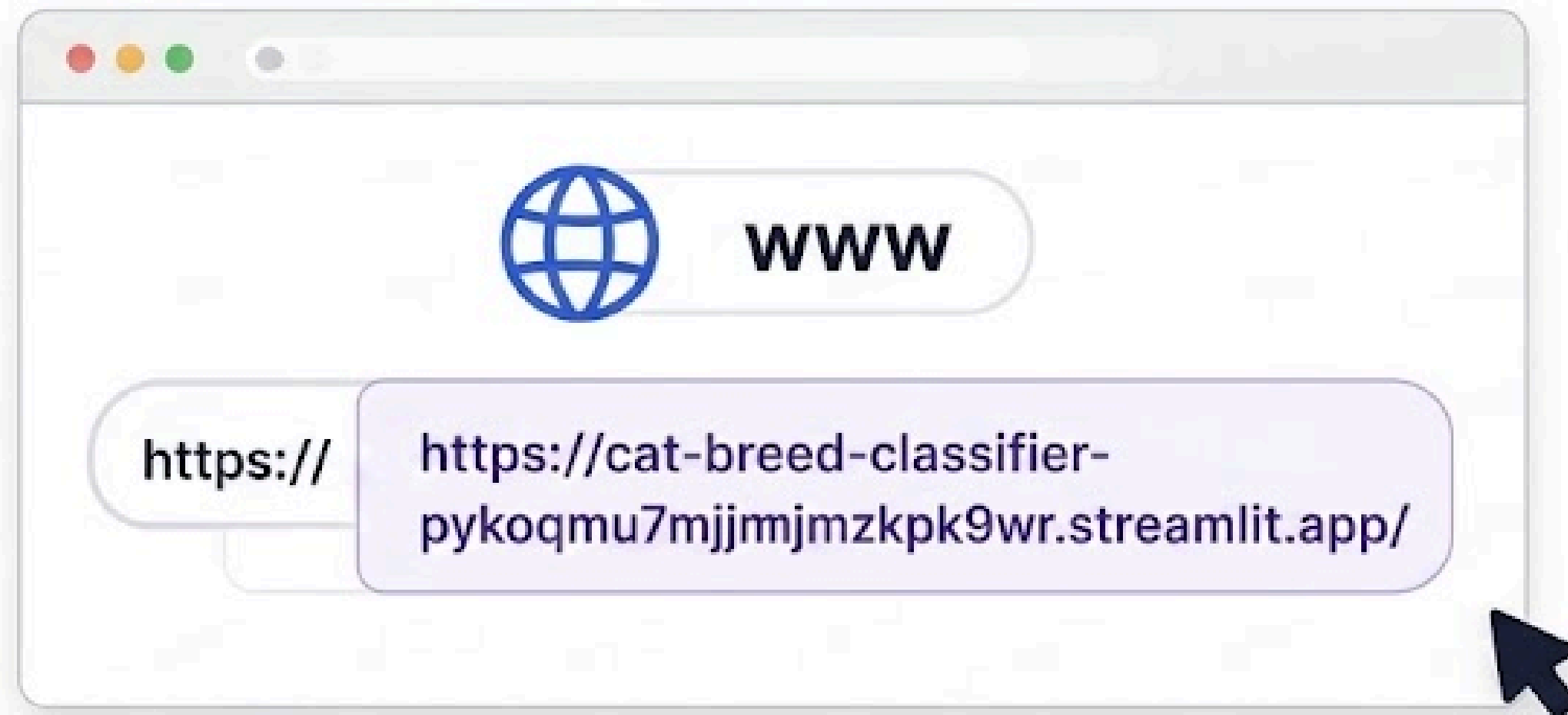
```
import torch

@st.cache_resource
def load_model():
    model = ConvNextTiny(num_classes=66)
    state_dict = torch.load(
        'cat_breed_model.pth',
        map_location='cpu'
    )
    model.load_state_dict(state_dict)
    model.eval()
    return model
```

8

URL สาธารณะ

แอปพร้อมใช้งานบน Streamlit Community Cloud



ใช้งานง่าย
ผ่าน Browser



ไม่ต้องติดตั้ง
โปรแกรมเพิ่ม



รองรับมือถือ
และแท็บเล็ต



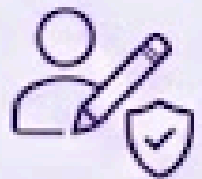
ใช้งานได้
ตลอดเวลา

9

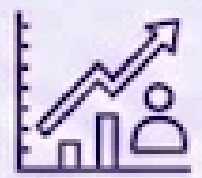
การดูแลรักษาและทดสอบระบบ



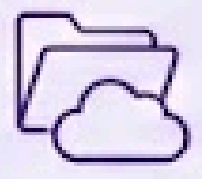
ตรวจสอบการทำงานของระบบเป็นประจำ



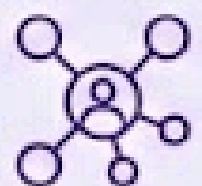
อัปเดตข้อมูลสายพันธุ์และแก้ไขข้อมูลให้ถูกต้อง



ติดตามประสิทธิภาพและการใช้งาน



สำรองไฟล์โมเดลและข้อมูลสำคัญ



พร้อมให้บริการอย่างต่อเนื่อง

